

BÀI THỰC HÀNH — BUỔI 5

Mục tiêu & đầu ra

Thiết kế 1 thành phần RAG Foundation độc lập, dùng kỹ thuật OCR để đọc PDF tiếng Việt (đặc biệt PDF scan), sau đó minh hoạ trực quan ba chiến lược chunking: fixed-size, semantic và hierarchical. Đầu ra là giúp người học nhìn thấy từng bước chuyển đổi từ trang PDF sang text rồi thành chunk và hiểu cách các chunk hoạt động.

Phần 0 — Khởi tạo cấu trúc thư mục

Thực hiện bước này trước khi viết SPEC hoặc code. Không tạo dữ liệu thật, không ghi đè thư mục hay file đã tồn tại.

```
[ROLE] Bạn là coding agent hướng dẫn người mới học RAG.
[GOAL] Kiểm tra cấu trúc hiện có rồi tạo cấu trúc bài làm độc lập theo thứ tự:
`RAG/`, `RAG/rag_foundation/`, `RAG/rag_foundation/buoi_05/`, và
`RAG/rag_foundation/buoi_05/datademo/`. Trong `datademo/` đặt các file PDF tiếng
Việt công khai từ ./datademo
[CONSTRAINTS] Không xoá, đổi tên hoặc ghi đè file/thư mục hiện có. Nếu tên đã tồn
tại, báo rõ và dùng lại sau khi kiểm tra. Chỉ tạo mã trong thư mục Buổi 5. Không
dùng tài liệu nội bộ hay dữ liệu nhạy cảm.
[OUTPUT] Cây thư mục thực tế, danh sách file đã tạo, nguồn/ghi chú rằng PDF là dữ
liệu công khai hoặc mô phỏng, và lệnh terminal phù hợp hệ điều hành để kiểm tra
lại cây thư mục.
```

Cấu trúc tối thiểu sau bước này:

```
RAG/
├── rag_foundation/
│   ├── buoi_05/
│   │   ├── datademo/
│   │   │   └── van_ban_mau.pdf
│   │   ├── src/
│   │   ├── storage/
│   │   └── tests/
```

Phần 1 — Prompt kiểm tra và cài đặt môi trường OCR

```
[ROLE] Bạn là Python developer chuyên xử lý văn bản tiếng Việt. Môi trường python
sẽ chọn là môi trường bạn tự động tạo sau đây.
[CONTEXT] Đọc cấu trúc thư mục Buổi 5 trước khi sửa.
[GOAL] Kiểm tra Python, PyMuPDF, Pillow, Llama_cloud, Pydantic, Streamlit, dotenv
nếu chưa có, thông báo cho người dùng trước khi agent tự tải/cài phần mềm hệ
thống. Tạo `src/check_ocr_env.py` để kiểm tra và in bảng PASS/FAIL.
```

[CONSTRAINTS] Không in secret, không sửa PDF gốc.
 [OUTPUT] Danh sách công cụ, kết quả kiểm tra agent đã thực hiện, code kiểm tra và agent tự khắc phục từng trạng thái FAIL bằng tiếng Việt dễ hiểu.

Tạo file môi trường: Tạo file .env trong buoi_05/src với nội dung sau: LLAMA_CLOUD_API_KEY='KEY CỦA BẠN'

Phần 2 — Agent Spec

Tạo RAG/rag_foundation/buoi_05/SPEC_buoi_05.md. SPEC phải nêu rõ đầu vào là PDF tiếng Việt trong datademo/; đầu ra gồm text OCR chuẩn Unicode NFC, metadata source, page, ocr_used, language, và báo cáo của ba chiến lược chunking. Xác định rõ ba cách cần so sánh:

- **Fixed-size:** cắt theo số ký tự/token với overlap.
- **Semantic:** ưu tiên ranh giới đoạn văn thường ngắt như ngắt đoạn, kết đoạn, cách dòng.
- **Hierarchical:** chia theo cấu trúc mà mỗi Chương → Mục → Điều/Khoản → Điểm sẽ thành mốc bắt đầu của 1 chunk. Nêu việc cần sử dụng key trong .env thuộc folder src nhưng không được phép đọc giá trị của các key. SPEC cũng phải quy định không tạo embedding, không lưu vector database và không gọi LLM trong Buổi 5, code ở mức demo đơn giản không phức tạp hóa, không bỏ sót yêu cầu.

Phần 3 — Prompt làm chức năng OCR và chunking

```
[ROLE] Bạn là Python RAG engineer và giáo viên.
[CONTEXT] Dùng `SPEC_buoi_05.md`. Bài làm chỉ nằm trong
`RAG/rag_foundation/buoi_05/`. API_KEY đã có sẵn trong .env
[GOAL] Viết luồng độc lập: (1) đọc PDF trong `datademo/`; (2) thử lấy text layer
bằng PyMuPDF; (3) khi trang không thể dùng pymupdf hoặc text tách từ pymupdf bị
lỗi(lỗi font, lỗi encoding, lỗi ký tự lạ, lỗi rỗng), render sang ảnh và gửi OCR
toàn bộ file bằng llamaparse từ llama-cloud.; (4) chuẩn hoá Unicode NFC; (5) Lưu
lại data ở dạng raw này vào một folder output; (6) Tạo code cho phép thử nghiệm
chiến thuật fixed-size, semantic: hết đoạn, cách dòng và hierarchical: tiêu đề,
mục lớn, nhỏ của trang. Mỗi chunk phải có `chunk_id`, `strategy`, `source`,
`page_start`, `page_end`, `text`, và metadata cấu trúc nếu có.
[CONSTRAINTS] Không tạo embedding, không gọi LLM, không ghi đè PDF gốc. Không bịa
heading khi PDF không có cấu trúc; phải ghi cảnh báo.
[OUTPUT] Danh sách file tạo/sửa, lệnh dry-run và lệnh `--write`, một ví dụ
metadata, thống kê số chunk/độ dài min-max-trung bình cho từng chiến lược, và ít
nhất 3 tình huống lỗi đã xử lý.
Cách gọi API Llamaparse:
from llama_cloud import AsyncLlamaCloud

client = AsyncLlamaCloud(api_key="")

file_obj = await client.files.create(file="./my_document.pdf", purpose="parse")

result = await client.parsing.parse(
    file_id=file_obj.id,
    tier="agentic",
    version='latest',
    expand=["markdown_full"],
)
```

```
print(result.markdown_full)
```

Phần 4 — Prompt review code

Review toàn bộ Buổi 5 theo SPEC. Kiểm tra RAG\rag_foundation\buoi_05\output: Llama parse có đang được gọi; PDF có text layer có tránh OCR không cần thiết; OCR có chạy khi text layer bị lỗi(lỗi font, lỗi encoding, ký tự lạ, rỗng); Unicode tiếng Việt có được chuẩn hoá NFC; fixed-size có overlap hợp lý; semantic không cắt giữa câu khi có thể; hierarchical không bịa cấu trúc; lỗi một trang không làm dừng job; PDF gốc và secret không bị ghi/log. Chỉ ra file/dòng cần sửa, sửa tối thiểu, chạy lại dry-run và báo bằng test input/expected/actual/PASS-FAIL.

Phần 5 — Prompt tạo UI Streamlit để trực quan hoá

Tạo `RAG/rag\_foundation/buoi\_05/app.py` bằng Streamlit. UI tiếng Việt để visualize chunk trong RAG\rag_foundation\buoi_05\output. Agent đưa lệnh khởi chạy UI.

Phần 6 — Dừng streamlit

Dừng quá trình streamlit đang chạy lại bằng cú pháp: Ctrl + C

Checklist

- ☐ Có cây thư mục `RAG/rag_foundation/buoi_05/datademo/` và một PDF tiếng Việt công khai/mô phỏng.
- ☐ OCR dùng Llaparse, có fallback và cảnh báo phù hợp.
- ☐ So sánh được fixed-size, semantic và hierarchical.
- ☐ UI Streamlit cho thấy PDF → OCR/text → chunk một cách trực quan.
- ☐ Chưa tạo vector database hoặc gọi LLM ở Buổi 5.